

Python Decorators

[< Previous](#)[Next >](#)

Decorators let you add extra behavior to a function, without changing the function's code.

A decorator is a function that takes another function as input and returns a new function.

Basic Decorator

Define the decorator first, then apply it with `@decorator_name` above the function.

Example

[Get your own Python Server](#)

A basic decorator that uppercases the return value of the decorated function.

```
def changecase(func):  
    def myinner():  
        return func().upper()  
    return myinner  
  
@changecase  
def myfunction():  
    return "Hello Sally"  
  
print(myfunction())
```

Try it Yourself »

By placing `@changeCase` directly above the function definition, the function `myfunction` is being "decorated" with the `changeCase` function.

The function `changeCase` is the decorator.

The function `myfunction` is the function that gets decorated.

Multiple Decorator Calls

A decorator can be called multiple times. Just place the decorator above the function you want to decorate.

Example

Using the `@changeCase` decorator on two functions:

```
def changeCase(func):
    def myinner():
        return func().upper()
    return myinner

@changeCase
def myfunction():
    return "Hello Sally"

@changeCase
def otherfunction():
    return "I am speed!"

print(myfunction())
print(otherfunction())
```

Try it Yourself »

LEARN P
GET CER
CHECK IT

w³
schools

COLOR
PICKER



Arguments in the Decorated Function

Functions that requires arguments can also be decorated, just make sure you pass the arguments to the wrapper function:

Example

Functions with arguments can also be decorated:

```
def changecase(func):  
    def myinner(x):  
        return func(x).upper()  
    return myinner  
  
@change-case  
def myfunction(nam):  
    return "Hello " + nam  
  
print(myfunction("John"))
```

Try it Yourself »

*args and **kwargs

Sometimes the decorator function has no control over the arguments passed from decorated function, to solve this problem, add (*args, **kwargs) to the wrapper function, this way the wrapper function can accept any number, and any type of arguments, and pass them to the decorated function.

Example

Secure the function with `*args` and `**kwargs` arguments:

```
def changecase(func):  
    def myinner(*args, **kwargs):  
        return func(*args, **kwargs).upper()  
    return myinner  
  
@change-case  
def myfunction(nam):  
    return "Hello " + nam  
  
print(myfunction("John"))
```

Try it Yourself »

Decorator With Arguments

Decorators can accept their own arguments by adding another wrapper level.

Example

A decorator factory that takes an argument and transforms the casing based on the argument value.

```
def changecase(n):  
    def changecase(func):  
        def myinner():  
            if n == 1:  
                a = func().lower()  
            else:  
                a = func().upper()  
            return a  
        return myinner  
    return changecase
```



Tutorials ▼

References ▼

Exercises ▼

Certificates ▼

Search



In

[HTML](#) [CSS](#) [JAVASCRIPT](#) [SQL](#) [PYTHON](#) [JAVA](#) [PHP](#) [HOW TO](#) [W3.CSS](#) [C](#) [C](#) >[Python Comments](#)[Python Variables](#)[Python Data Types](#)[Python Numbers](#)[Python Casting](#)[Python Strings](#)[Python Booleans](#)[Python Operators](#)[Python Lists](#)[Python Tuples](#)[Python Sets](#)[Python Dictionaries](#)[Python If...Else](#)[Python Match](#)[Python While Loops](#)[Python For Loops](#)[Python Functions](#)[Python Decorators](#)[Python Range](#)[Python Lambda](#)[Python Arrays](#)[Python OOP](#)[Python Classes/Objects](#)[Python Inheritance](#)[Python Iterators](#)[Python Polymorphism](#)[Python Scope](#)[Python Modules](#)[Python Dates](#)[Python Math](#)[Python JSON](#)[Python RegEx](#)[Python PIP](#)[Python Try...Except](#)

```
print(myfunction())
```

[Try it Yourself »](#)

Multiple Decorators

You can use multiple decorators on one function.

This is done by placing the decorator calls on top of each other.

The decorators are called in the order they are specified.

Example

One decorator for upper case, and one for adding a greeting:

```
def changecase(func):
    def myinner():
        return func().upper()
    return myinner

def addgreeting(func):
    def myinner():
        return "Hello " + func() + " Have a good day!"
    return myinner

@changecase
@addgreeting
def myfunction():
    return "Tobias"

print(myfunction())
```

[Try it Yourself »](#)

Preserving Function Metadata

Functions in Python has metadata that can be accessed using the `__name__` and `__doc__` attributes.

Example

Normally, a function's name can be returned with the `__name__` attribute:

```
def myfunction():  
    return "Have a great day!"  
  
print(myfunction.__name__)
```

[Try it Yourself »](#)

But, when a function is decorated, the metadata of the original function is lost.

Example

Try returning the name from a decorated function and you will not get the same result:

```
def changecase(func):  
    def myinner():  
        return func().upper()  
    return myinner  
  
@changecase  
def myfunction():
```

```
    return "Have a great day!"  
  
print(myfunction.__name__)
```

Try it Yourself »

To fix this, Python has a built-in function called `functools.wraps` that can be used to preserve the original function's name and docstring.

Example

Import `functools.wraps` to preserve the original function name and docstring.

```
import functools  
  
def changecase(func):  
    @functools.wraps(func)  
    def myinner():  
        return func().upper()  
    return myinner  
  
@changease  
def myfunction():  
    return "Have a great day!"  
  
print(myfunction.__name__)
```

Try it Yourself »

Exercise ?

What is the main purpose of a decorator in Python?